

✓ 1. Evaluación de tu código

Tu programa **funciona bien** en estructura y cumple los requisitos del enunciado:

Funcionalidad	Estado	Comentario
Añadir alumnos	✓	Usa <code>BufferedWriter</code> con <code>FileWriter(..., true)</code> para modo append. Bien hecho.
ID autonumérico	✓	Calculado con <code>obtenerUltimoId()</code> . Correcto.
Buscar alumno por nombre/apellidos	✓	Usa <code>BufferedReader</code> y split por "-". Lógico y funcional.
Añadir notas	✓	Usa <code>BufferedWriter</code> en modo append. Cumple el enunciado.
Calcular media	✓	Suma todas las notas del alumno y saca la media. Correcto.
Control de excepciones	⚠	Bien manejadas, pero podrías definir una excepción personalizada para el caso del alumno no encontrado.
Formato del archivo	⚠	El enunciado pide <code>***</code>

🧠 2. Métodos y clases clave para aprobar con seguridad

Estas son las **clases más importantes del examen** y sus **usos típicos**:

Clase	Uso principal	Ejemplo
<code>File</code>	Representa un fichero o carpeta. Permite crear, verificar si existe, etc.	<code>File archivo = new File("Alumnos.txt");</code>
<code>FileWriter</code>	Escribe texto en un archivo (sobrescribe).	<code>new FileWriter(archivo);</code>
<code>BufferedWriter</code>	Escribe texto de forma eficiente (en memoria intermedia). Se usa junto a <code>FileWriter</code> .	<code>new BufferedWriter(new FileWriter(archivo, true));</code>

<code>FileReader</code>	Lee caracteres de un archivo.	<code>new FileReader(archivo);</code>
<code>BufferedReader</code>	Lee líneas completas de texto de forma eficiente.	<code>BufferedReader br = new BufferedReader(new FileReader(archivo));</code>
<code>Scanner</code>	Para leer datos desde teclado.	<code>Scanner sc = new Scanner(System.in);</code>
<code>Exception</code>	Controla errores de E/S (entrada/salida).	<code>catch (IOException e) { ... }</code>

Métodos clave que debes dominar

Método o bloque	Qué hace	Cuándo usar
<code>new FileWriter(ruta, true)</code>	Abre en modo append (no borra contenido).	Para añadir líneas nuevas.
<code>BufferedWriter.write()</code>	Escribe una línea o cadena.	Para guardar datos en un fichero.
<code>BufferedReader.readLine()</code>	Lee línea por línea.	Para buscar o procesar datos almacenados.
<code>String.split(String regex)</code>	Divide una línea en campos.	Para separar los datos por delimitadores (`
<code>Double.parseDouble()</code>	Convierte texto a número decimal.	Para calcular medias, notas, etc.
<code>Integer.parseInt()</code>	Convierte texto a entero.	Para procesar IDs.
<code>try-with-resources</code>	Cierra automáticamente los ficheros.	Recomendado en producción. Ejemplo:

```
try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
    ...
}
```



3. Pasos y claves para acceder y manipular ficheros

Estos son los **5 pasos esenciales** que deberías tener **muy claros** para el examen:

♦ 1. Comprobar o crear el fichero

```
File f = new File("Alumnos.txt");
if (!f.exists()) {
    f.createNewFile();
}
```

♦ 2. Escribir datos

- Usa `FileWriter` + `BufferedWriter`
- Abre en modo *append* (`true`)
- Escribe con formato consistente (`|`, `;`, `-`, etc.)

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("Alumnos.txt", true)))
    bw.write("1|Pepe|Perez|01-01-2000|1º DAM;");
    bw.newLine();
}
```

♦ 3. Leer datos

- Usa `FileReader` + `BufferedReader`
- Recorre con `while((linea = br.readLine()) != null)`

```
try (BufferedReader br = new BufferedReader(new FileReader("Alumnos.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
}
```

♦ 4. Buscar información

- Divide cada línea con `split()`
- Compara campos

```
String[] partes = linea.split("\\|");
if (partes[1].equals(nombre) && partes[2].equals(apellidos)) {
    id = Integer.parseInt(partes[0]);
}
```

♦ 5. Procesar y calcular datos

- Convierte los datos leídos (`String`) a su tipo (`int`, `double`).
- Usa acumuladores (`suma`, `contador`, etc.) para cálculos.

2.

quiero que me des consejos sobre la creacion de metodos para que se encargue exclusivamente de realizar la tarea concreta, como la de buscar id para comprobar si el alumno existe y retorne el nºid, la de insercion de notas si encuentra el alumno, calcular la media de notas. Quiero que me digas si el codigo que hay en el switch es el correcto y luego el que se realiza en el metodo en concreto también se hace correcta.

Perfecto 🙌, esto es exactamente lo que diferencia a un **programador que aprueba el examen “justo”** de uno que lo **aprueba con sobresaliente**: la **estructura modular** y la **claridad en la responsabilidad de cada método**.

Vamos a analizar tu código desde esa perspectiva:

- Cómo está estructurado el `switch`.
- Qué hace bien y qué podrías mejorar.
- Qué principios seguir para crear métodos claros, reutilizables y bien definidos.
- Revisión concreta de cada método (`devolverId`, `insertarNotas`, `calcularMedia`).

1. El propósito de dividir en métodos

Cada método debe:

- **Hacer una sola cosa** (“single responsibility”).
- **Tener un nombre descriptivo** del tipo **verbo + objeto** (`buscarAlumnoPorId`, `insertarNotas`, `calcularMedia...`).
- **Recibir solo los parámetros necesarios** (no más, no menos).
- **Devolver un valor útil** o realizar una acción clara.

👉 Esto hace que el `switch` quede limpio y **fácil de leer**, y cada bloque de código (método) se pueda **reutilizar y probar por separado**.

2. Análisis del `switch` principal

Tu `switch` está muy bien estructurado.

Cada `case` **llama a un método** que encapsula una funcionalidad concreta, y eso es justo lo que se espera en un examen.

Veámoslo con comentarios y mejoras:

```
switch (opcion) {
```

```
    case 1:
```

```
        // ✅ Correcto: método separado que se encarga de añadir alumnos.
```

```
        anadirAlumno();
```

```
        break;
```

```
    case 2:
```

```
// ✔ Correcto: lee nombre/apellidos y llama a devolverId()

System.out.println("Nombre:");

String nombreBuscado = sc.nextLine();

System.out.println("Apellidos: ");

String apellidosBuscado = sc.nextLine();


int id = devolverId(nombreBuscado, apellidosBuscado);

if (id != 0) { // ⚠ Mejor usar -1 como valor "no encontrado" (más claro)

    System.out.println("El alumno " + nombreBuscado + " " + apellidosBuscado + " tiene
el ID: " + id);

}

break;
```

case 3:

```
// ✔ Correcto: busca ID antes de insertar notas.
```

```
System.out.println("Nombre:");

String nombreAlumno = sc.nextLine();

System.out.println("Apellidos: ");

String apellidosAlumno = sc.nextLine();
```

```
int idAlumno = devolverId(nombreAlumno, apellidosAlumno);

if (idAlumno != 0) {

    insertarNotas(idAlumno);

}

break;
```

case 4:

```

// ✔ Correcto: reutiliza devolverId para luego calcular media.

System.out.println("Nombre:");

String nombreMedia = sc.nextLine();

System.out.println("Apellidos: ");

String apellidosMedia = sc.nextLine();

int idMedia = devolverId(nombreMedia, apellidosMedia);

if (idMedia != 0) {

    calcularMedia(idMedia);

}

break;

case 0:

    System.out.println("Saliendo del programa.....");

    break;

default:

    System.out.println("Introduzca un valor correcto");

}

```

✔ Conclusión:

El **switch** es **perfectamente correcto**.

Cumple con la buena práctica de **delegar responsabilidades** a métodos específicos.

Solo puedes mejorar usando valores como **-1** para indicar “no encontrado”, y quizá mover las preguntas de nombre/apellidos a un método auxiliar.

3. Evaluación de los métodos clave

Vamos a revisar los tres más importantes uno por uno.

✂ devolverId(String nombre, String apellidos)

Propósito: Buscar un alumno en el archivo y devolver su ID.

Tu versión:

✓ Bien hecha, recorre el archivo con `BufferedReader` y compara nombre/apellidos.

⚠ Mejora menor: usa un valor de retorno `-1` si no se encuentra, y `split("\\|")` en lugar de `" - "` para coincidir con el formato del examen.

Versión mejorada:

```
private static int devolverId(String nombre, String apellidos) {

    File ruta = new File("ficheros/RepasoExamenFicheros/Alumnos.txt");

    int id = -1;

    try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {

        String linea;

        while ((linea = br.readLine()) != null) {

            String[] partes = linea.trim().split("\\|");

            if (partes.length >= 3 &&

                nombre.equalsIgnoreCase(partes[1].trim()) &&

                apellidos.equalsIgnoreCase(partes[2].trim())) {

                id = Integer.parseInt(partes[0].trim());

                break;

            }

        }

    } catch (IOException e) {

        System.out.println("Error al leer el archivo: " + e.getMessage());

    }

    if (id == -1) {
```



```
        System.out.println("El alumno con nombre " + nombre + " y apellido " + apellidos + " no  
se encuentra en el archivo.");
```

```
    }
```

```
    return id;
```

```
}
```

insertarNotas(int id)

Propósito: Añadir notas a un alumno si existe.

Tu versión:

✓ Correcta, usa `BufferedWriter` en modo `append`.

⚠ Mejora: Añadir una separación (`newLine()`) y validar el formato de las notas.

Versión mejorada:

```
private static void insertarNotas(int id) {  
  
    File notas = new File("ficheros/RepasoExamenFicheros/notas.txt");  
  
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(notas, true));  
  
        Scanner sc = new Scanner(System.in)) {  
  
        System.out.println("Introduzca notas (separadas por ;): ");  
  
        String calificaciones = sc.nextLine();  
  
        bw.write(id + "|" + calificaciones);  
  
        bw.newLine();  
  
        System.out.println("Notas añadidas correctamente al alumno con ID " + id);  
  
    } catch (IOException e) {  
  
        System.out.println("Error al escribir notas: " + e.getMessage());  
  
    }  
  
}
```

✖ calcularMedia(int id)

Propósito: Calcular la media de todas las notas del alumno en `notas.txt`.

Tu versión:

- ✓ Bien estructurada.
- ⚠ Pequeñas mejoras:

- Comprobar si el alumno tiene notas antes de dividir.
- Usar `try-with-resources`.
- Usar `split("\\|")` para coherencia.

Versión mejorada:

```
private static void calcularMedia(int id) {  
  
    File notas = new File("ficheros/RepasoExamenFicheros/notas.txt");  
  
    double suma = 0;  
  
    int numeroNotas = 0;  
  
  
    try (BufferedReader br = new BufferedReader(new FileReader(notas))) {  
  
        String linea;  
  
        while ((linea = br.readLine()) != null) {  
  
            String[] partes = linea.trim().split("\\|");  
  
            if (partes[0].equals(String.valueOf(id))) {  
  
                String[] notasAlumno = partes[1].split(";");  
  
                for (String notaStr : notasAlumno) {  
  
                    suma += Double.parseDouble(notaStr);  
  
                    numeroNotas++;  
  
                }  
  
            }  
  
        }  
  
    }  
}
```

```
if (numeroNotas > 0) {  
    double media = suma / numeroNotas;  
  
    System.out.printf("La media del alumno con ID %d es %.2f%n", id, media);  
} else {  
    System.out.println("El alumno con ID " + id + " no tiene notas registradas.");  
}  
  
} catch (IOException e) {  
    System.out.println("Error al leer notas: " + e.getMessage());  
}  
}
```



4. Claves para diseñar buenos métodos en exámenes

Clave	Qué significa	Ejemplo
1. Nombre claro	Usa verbos que digan lo que hace.	<code>buscarAlumnoPorId,</code> <code>insertarNotas,</code> <code>calcularMedia.</code>
2. Una tarea por método	No mezcles lógica (leer + calcular + escribir).	Divide en pasos.
3. Evita repetir código	Usa métodos auxiliares (por ejemplo, <code>devolverId()</code>).	En varios <code>case</code> .

4. Usa retornos útiles	Que devuelvan datos que se puedan usar.	<code>int devolverId()</code> devuelve un número, no imprime.
5. Controla excepciones	No dejes que el programa se caiga.	<code>try-catch</code> o <code>throws</code> .
6. Coherencia de formato	Usa los mismos delimitadores en todos los ficheros.	`

En resumen

✓ Tu `switch` está bien estructurado.

✓ Cada método hace su tarea concreta y está bien planteado.

💡 Solo necesitas mejorar formato, valores de retorno y el uso de

`try-with-resources`.

💪 Si entiendes bien:

- `BufferedReader` / `BufferedWriter`
 - `split()`
 - Búsqueda en ficheros línea a línea
entonces tienes **todas las herramientas para aprobar este examen sin problema.**
-

Si quieres, te puedo preparar una **versión final lista para examen**, con nombres de métodos más claros, comentarios explicativos y formato idéntico al del enunciado (`|` y `;`).
¿Quieres que te la deje lista y comentada para repasar antes del examen?

Consejos finales para el examen

1. **Usa el formato del enunciado** (con `|` y `;`).
2. **Controla las rutas** de los ficheros (usa rutas relativas simples como `"Alumnos.txt"` si puedes).
3. **Cierra siempre los flujos** (`close()` o `try-with-resources`).
4. **Maneja las excepciones** con mensajes claros.
5. **Comenta el código** si te sobra tiempo: `// Lee alumnos del archivo`, etc.
6. **Haz pruebas con varios alumnos** y asegúrate de que el ID se incrementa correctamente.

3.

quiero que me digas si el menu esta bien , y el metodo de agregar alumno tambien, dime tambien si la forma de saber el ultimo id del documento es la acertada.

Perfecto 👍 — vamos a revisar en **detalle profesional** las tres partes que mencionas:

- 1 El menú principal (**switch**)
- 2 El método **anadirAlumno()**
- 3 El método **obtenerUltimoId()**

Te voy a decir con total claridad qué está bien, qué puede mejorarse y cómo hacerlo correctamente **de cara a examen**.

1 MENÚ PRINCIPAL

Tu menú actual es **correcto y funcional**.

Veamos tu código base (resumido):

```
do {
    System.out.println("-----MENU-----\n" +
        "1 - Anadir alumnos\n" +
        "2 - Devolver ID por nombre y apellido\n" +
        "3 - Insertar notas\n" +
        "4 - Calcular media notas alumno\n" +
        "0 - Salir\n" +
        "-----)\n" +
        "INGRESE UNA OPCION: ");
    opcion = sc.nextInt();
    sc.nextLine();

    switch (opcion) {
        case 1: anadirAlumno(); break;
        case 2: ...
        case 3: ...
        case 4: ...
        case 0: System.out.println("Saliendo..."); break;
        default: System.out.println("Introduzca un valor correcto");
    }
} while (opcion != 0);
```

✓ Lo que está bien:

- ✓ Menú limpio y claro.
- ✓ Uso correcto de **Scanner** y **sc.nextLine()** para limpiar el buffer.
- ✓ Cada caso llama a un método, lo cual es buena práctica.
- ✓ Buena estructura de bucle **do-while**: mantiene el menú activo hasta elegir salir.

⚠ Lo que puedes mejorar:

1. **El formato visual**: quita el paréntesis de cierre en la línea del menú (sobrante).
2. **Usar un método auxiliar** para pedir nombre y apellidos (para no repetir 3 veces el mismo código).

Ejemplo de método auxiliar:

```
private static String[] pedirNombreYApellidos(Scanner sc) {  
    System.out.println("Nombre:");  
    String nombre = sc.nextLine();  
    System.out.println("Apellidos:");  
    String apellidos = sc.nextLine();  
    return new String[]{nombre, apellidos};  
}
```

Y en el menú:

case 2:

```
String[] datos = pedirNombreYApellidos(sc);  
int id = devolverId(datos[0], datos[1]);  
...
```

👉 Conclusión:

Tu menú está **correctamente estructurado**, es **válido para examen**, y solo requiere pulir pequeños detalles estéticos o de reutilización.

🧩 2 MÉTODO **anadirAlumno()**

Tu versión:

```

public static void anadirAlumno() {
    File archivo = new File("ficheros/RepasoExamenFicheros/Alumnos.txt");
    Scanner ss = new Scanner(System.in);
    int contador = obtenerUltimold("ficheros/RepasoExamenFicheros/Alumnos.txt");
    int anadirMas = 1;

    do {
        System.out.println("Nombre: ");
        String nombre = ss.nextLine();
        System.out.println("Apellidos: ");
        String apellidos = ss.nextLine();
        System.out.println("Fecha de nacimiento (dd/mm/aaaa)");
        String fecha = ss.nextLine();
        System.out.println("Clase: ");
        String clase = ss.nextLine();

        try {
            BufferedWriter bw = new BufferedWriter(new FileWriter(archivo, true));
            bw.write(contador + "-" + nombre + "-" + apellidos + "-" + fecha + "-" + clase + "\n");
            System.out.println("Alumno con ID " + contador + " anadido");
            bw.close();
            contador++;
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Pulse 1 para anadir alumno, pulse 0 para salir: ");
        anadirMas = ss.nextInt();
        ss.nextLine();
    } while (anadirMas == 1);
}

```

✓ Lo que haces bien:

- ✓ El método se encarga **únicamente de añadir alumnos** → ✓ responsabilidad única.
- ✓ Usas `BufferedWriter` con `FileWriter(..., true)` → ✓ modo *append*.
- ✓ Incrementas el ID después de cada inserción → correcto.
- ✓ Controlas bien el bucle de repetición → el usuario puede seguir insertando.

⚠ Detalles a mejorar:

1. **Formato del fichero:**

El enunciado pide | como separador y ; al final.

Tu código usa - y no añade el ;.

👉 En examen, **esto restaría puntos** si el profe corrige el formato exacto.

2. **Uso de try-with-resources:**

Así evitas olvidarte del close() (más limpio y seguro).

3. **Si el fichero no existe**, debería crearse automáticamente (lo hace `FileWriter`, pero puedes ser explícito).

4. **Claridad en los mensajes de consola.**

✅ **Versión mejorada (idéntica en funcionalidad, pero más correcta para examen):**

```
public static void anadirAlumno() {
    File archivo = new File("ficheros/RepasoExamenFicheros/Alumnos.txt");
    Scanner sc = new Scanner(System.in);
    int id = obtenerUltimold(archivo.getPath());
    int continuar;

    do {
        System.out.println("Nombre: ");
        String nombre = sc.nextLine();
        System.out.println("Apellidos: ");
        String apellidos = sc.nextLine();
        System.out.println("Fecha de nacimiento (dd-mm-aaaa): ");
        String fecha = sc.nextLine();
        System.out.println("Clase: ");
        String clase = sc.nextLine();

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(archivo, true))) {
            bw.write(id + "|" + nombre + "|" + apellidos + "|" + fecha + "|" + clase + ";");
            bw.newLine();
            System.out.println("Alumno con ID " + id + " añadido correctamente.");
            id++;
        } catch (IOException e) {
            System.out.println("Error al escribir en el archivo: " + e.getMessage());
        }

        System.out.println("Pulse 1 para añadir otro alumno, 0 para salir:");
        continuar = sc.nextInt();
        sc.nextLine(); // limpia buffer
    } while (continuar == 1);
}
```

}

👉 Conclusión:

Tu método está **bien diseñado conceptualmente**.

Solo fallas en el **formato del fichero** y podrías usar **try-with-resources** para hacerlo más limpio.



3 MÉTODO `obtenerUltimoId()`

Tu versión:

```
private static int obtenerUltimoId(String ruta) {
    File archivo = new File(ruta);
    int contador = 0;
    try {
        BufferedReader br = new BufferedReader(new FileReader(archivo));
        String linea;
        while ((linea = br.readLine()) != null){
            contador++; //CUENTA LAS LINEAS QUE HAY
        }
    } catch (Exception e) {
        e.getMessage();
    }
    return contador+1; // DEVUELVE EL SIGUIENTE ID PARA EL SIGUIENTE ALUMNO
}
```



Lo que haces bien:

- ✓ Recorres el fichero correctamente.
- ✓ Devuelves el siguiente ID (último + 1).
- ✓ Si el fichero está vacío, comienzas en 1.



Lo que se puede mejorar:

- Si alguna línea está vacía o mal escrita, el conteo puede ser erróneo.
→ Lo ideal es **leer el último ID directamente de la última línea**, no contar líneas.
- Debes cerrar el `BufferedReader` (o usar `try-with-resources`).

3. El `catch` no hace nada útil; mejor imprimir el error o tratar el caso del fichero inexistente.
4. Si el fichero no existe, `contador` debería empezar en 1.

✓ Versión mejorada (más segura y precisa):

```
private static int obtenerUltimold(String ruta) {
    File archivo = new File(ruta);
    int ultimold = 0;

    // Si el archivo no existe o está vacío, el primer ID será 1
    if (!archivo.exists()) return 1;

    try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
        String linea;
        String ultimaLinea = null;
        while ((linea = br.readLine()) != null) {
            if (!linea.trim().isEmpty()) {
                ultimaLinea = linea;
            }
        }

        if (ultimaLinea != null) {
            // Ejemplo de línea: 3|Jorge|Garcia|03-03-2000|1º SMR;
            String[] partes = ultimaLinea.split("\\|");
            ultimold = Integer.parseInt(partes[0]);
        }
    } catch (IOException | NumberFormatException e) {
        System.out.println("Error al obtener último ID: " + e.getMessage());
    }

    return ultimold + 1;
}
```

👉 Ventajas:

- No depende del número de líneas (más preciso).
 - Soporta líneas vacías o comentarios.
 - Devuelve el ID correcto aunque haya errores intermedios.
-

CONCLUSIÓN GENERAL

Componente	Veredicto	Comentario
 Menú principal	 Correcto	Estructura ideal, clara y modular.
 <code>anadirAlumno()</code>	 Bien planteado	Cumple su función, solo mejorar formato (
 <code>obtenerUltimoId()</code>	 Funcional, pero mejorable	Cuenta líneas en lugar de leer el último ID real. Mejor hacerlo con el último registro.

Claves para examen:

- 1 método = 1 responsabilidad.
 - Formato de archivo exacto al enunciado.
 - Siempre cerrar ficheros (**try-with-resources**).
 - Evitar repetir código (usa métodos auxiliares).
 - Controlar excepciones con mensajes claros.
-

creame un metodo para pedir nombre y apellidos para reutilizar codigo en los metodos 2, 3 y 4

Método `pedirNombreYApellidos()`

Objetivo:

Evitar repetir el mismo bloque:

```
System.out.println("Nombre:");
String nombre = sc.nextLine();
System.out.println("Apellidos:");
String apellidos = sc.nextLine();
```

en varios métodos (devolverId, insertarNotas, calcularMedia...).

✓ Implementación recomendada:

```
public static String[] pedirNombreYApellidos(Scanner sc) {  
    System.out.print("Nombre: ");  
    String nombre = sc.nextLine().trim();  
    System.out.print("Apellidos: ");  
    String apellidos = sc.nextLine().trim();  
    return new String[]{nombre, apellidos};  
}
```

🧠 Cómo usarlo:

- ♦ En el método 2 (devolver ID por nombre y apellidos):

```
case 2:  
    String[] datos = pedirNombreYApellidos(sc);  
    String nombre = datos[0];  
    String apellidos = datos[1];  
    devolverId(nombre, apellidos);  
    break;
```

- ♦ En el método 3 (insertar notas):

```
case 3:  
    String[] datos = pedirNombreYApellidos(sc);  
    String nombre = datos[0];  
    String apellidos = datos[1];  
    insertarNotas(nombre, apellidos);  
    break;
```

- ♦ En el método 4 (calcular media):

```
case 4:  
    String[] datos = pedirNombreYApellidos(sc);  
    String nombre = datos[0];  
    String apellidos = datos[1];  
    calcularMedia(nombre, apellidos);  
    break;
```

✓ Ventajas:

- Evitas **duplicar código**.
- Si quieres cambiar el formato (por ejemplo, validar que no estén vacíos), solo lo haces en un lugar.
- Mejora la **claridad y mantenimiento** del proyecto.

¿Quieres que te añada también una **validación básica** (por ejemplo, que no deje nombre/apellidos vacíos y los repita hasta que sean correctos)?

Es muy común que lo pidan en exámenes o prácticas para demostrar control de entrada de datos.